

Scam Sentinel — 전체 설계 & 작업 정리

△ ARCHIVED SNAPSHOT (frozen as of 2026-05-11)

*This document captures the design and work log **before** the QLoRA fine-tuned model shipped on 2026-05-12 and was deployed as the single production reasoner on 2026-05-14. Several claims below — production = Gemma 3 4B + Gemma 4 9B cascade, fine-tuning is 0%, RAG is wired into production, etc. — describe that earlier state and are **no longer accurate**.*

*For the current production architecture see [../README.md](#) and [diagrams/01_system_overview.mmd](#). The short version: every request now goes straight to a fine-tuned Gemma 4 E2B + QLoRA (*gemma4-sc* Q4_K_M GGUF via Ollama, F1 86.1% / FPR 1.1% on the 300-sample real test set), no Stage 1/ Stage 2 cascade, RAG off by default.*

*The body below is kept verbatim as a record of the pre-QLoRA design — it is useful context for the writeup but **must not be cited as the current state**.*

프로젝트: Gemma 4 Good Hackathon (마감 2026-05-18) 문서 작성일: 2026-05-11 작성 기간: 2026-05-10 ~ 2026-05-11 대화 요약

핵심 메시지 (모든 결정의 기준점)

"Scam Sentinel is not a final forensic deepfake detector. It is a multimodal scam-risk assistant that combines phone call transcript analysis, conversation patterns, retrieved real cases, and verification workflows — and improves itself from user feedback."

이 한 문장에서 추적되지 않는 기능은 MVP에 포함하지 않습니다.

사용자가 받는 4가지 질문에 대한 답

모든 의심스러운 입력에 대해 Scam Sentinel은 70대 어르신도 20대도 5초 안에 행동할 수 있는 평이한 언어로 답합니다:

1. **Is this whole situation a scam?** (이게 사기인가요?)
2. **Why is it dangerous?** (왜 위험한가요?)
3. **What should I do right now?** (지금 뭘 해야 하나요?)
4. **How do I verify with my family?** (가족과 어떻게 확인 하나요?)

1. 전체 아키텍처

1.1 시스템 개요 (한 화면 지도)



1.2 왜 하이브리드 캐스케이드인가

단계	모델	역할	평균 응답
Stage 1	Gemma 3 4B	빠른 분류 (safe? non-safe?)	~1초
Stage 2	Gemma 4 9B	깊은 추론 + 12 톨 호출	10~30초

300개 실제 평가에서 발견한 핵심 사실:

- Gemma 3 4B가 분류 정확도가 더 높음 (F1 80.8% vs 63.4%)
- Gemma 4 9B는 설명 품질 + 톨 호출이 더 좋음
- 각자 잘하는 일을 시키는 게 최적

일반 메시지의 대부분은 safe → Gemma 3에서 short-circuit → 1초 미만 응답. 의심스러운 메시지만 Gemma 4까지 도달.

2. 현재 데이터셋 카운트 (2026-05-10 확인)

파일	갯수	용도
<code>data/seeds.jsonl</code>	80	8개 카테고리 × 10개 손으로 쓴 시드
<code>data/seeds_real.jsonl</code>	571	UCI SMS Spam에서 분류한 실제 스팸
<code>data/synthetic/raw.jsonl</code>	1,112	시드 기반 Gemma 4 증식 (×13)
<code>data/synthetic/raw_real.jsonl</code>	2,224	실제 스팸 기반 Gemma 4 증식 (×4)
<code>data/synthetic/combined.jsonl</code>	3,907	위 3개 합친 원본
(필터링 후)	3,871	중복/메타단어/비현실 제거
<code>data/synthetic/train.jsonl</code>	3,100	80% 층화 분할 — 옵션 파인튜닝용
<code>data/synthetic/dev.jsonl</code>	771	20% 층화 분할 — early stopping용
<code>data/evaluation/eval_set.jsonl</code>	300	실제 라벨 손작업, 학습에 절대 안 씀
<code>data/evaluation/eval_set_70.jsonl</code>	70	원래 70개셋 아카이브
<code>data/rag_cases.jsonl</code>	117	FTC/APWG 실제 사례 (RAG용, 목표 150~200)
<code>data/vector_store/</code>	(ChromaDB)	117개 RAG 사례의 영구 벡터 인덱스

data/user_feedback.jsonl	9 (만약금)	Self-improving Cascade 업적
--------------------------	---------	---------------------------

8개 카테고리: family_impersonation, prosecutor_scam, bec_scam, romance_scam, package_scam, bank_phishing, phishing_link, normal

2.1 테스트셋 (200개) 구성

--	--	--	--

- 위험도 라벨: safe 175 / low 7 / medium 79 / high 26 / critical 13
- 카테고리: normal 179 / phishing_link 88 / family_impersonation 8 / prosecutor_scam 6 / bec_scam 5 / romance_scam 5 / bank_phishing 5 / package_scam 4

3. 마지막 평가 결과 (300개 실제 테스트셋, 2026-05-06)

Setup	Accuracy	Precision	Recall	F1	FPR
gemma3:4b / v3 / no RAG ✓ 프로덕션	80.3%	68.1%	99.2%	80.8%	33.1%
gemma3:4b / v3 / + RAG	65.7%	54.8%	100.0%	70.8%	58.9%
gemma4 9B / v3 / no RAG	53.0%	46.9%	97.6%	63.4%	78.9%

3.1 70개 → 300개 확장이 바꾼 결론

모델 설정	70개 F1 / FPR	300개 F1 / FPR	FPR 변화
gemma3:4b / v2 / no RAG	92.8% / 28.0%	80.8% / 33.1% (v3)	+5.1 pt
gemma3:4b / v2 / + RAG	91.5% / 24.0%	70.8% / 58.9% (v3)	+34.9 pt
gemma4 9B / v3 / no RAG	83.3% / 72.0%	63.4% / 78.9%	+6.9 pt

3.2 4가지 핵심 발견

1. 작은 모델이 분류는 더 잘함. Gemma 4 9B의 "돈/긴급 단어 → low risk" 캘리브레이션 편향이 300개 셋에서 드러남.
2. **RAG**가 오히려 해가 됨. 일반 ham 메시지에 사기 사례가 잘못 매칭되어 오탐 폭발 (FPR +34.9pt).
3. 재현율은 다 **97~100%**. 진짜 사기는 모두 잡음. 문제는 오탐.
4. 프로덕션: gemma3:4b without RAG (분류 + 톨 추론) + gemma4 9B (설명 + CoT 추론).

4. 파인튜닝 현황 (중요)

4.1 현재 상태: 파인튜닝 0%, 사전학습 모델 그대로 사용

질문	답
Train(3,100)으로 모델 훈련시켰나?	✗ 아직 안 함
모델 파라미터 바꿨?	✗ 안 바꿨
Dev(771)로 최적화 모델 찾음?	✗ 안 함
현재 어떤 모델을 쓰는가?	Ollama로 받은 Gemma 3 4B + Gemma 4 9B 사전학습 그대로

4.2 왜 안 했나 (CLAUDE.md §12)

1. 현재 **baseline F1 80.8%**로 충분히 좋음 — 파인튜닝 기대 이득 3~5%p이지만 리스크가 큼
2. **catastrophic forgetting** 위험: 일반 추론 능력 잃을 수 있음
3. 합성 데이터 **overfit** 위험: 실제 평가셋 점수 떨어질 가능성
4. **JSON** 출력 형식 깨질 위험: function calling 자체 망가짐
5. **16일** 안에 디버깅 시간 못 뺌

4.3 미래 계획: Track B — Colab Pro L4 + 4-bit QLoRA

- Colab Pro \$9.99/월 + L4 GPU (22.5GB VRAM)
- Unsloth + TRL의 SFTTrainer or DPOTrainer
- LoRA r=16, alpha=32, target=q/k/v/o
- Gemma 3 4B 기준: 4-bit 모델 ~2.5GB + LoRA + grad ~4GB = ~6.5GB
- 어댑터 크기 ~50MB (base 모델의 1/180)
- **base** 가중치는 절대 안 바꿈, **LoRA** 어댑터만 **swap** → 즉시 롤백 가능

가이드: [docs/colab-finetuning-guide.md](https://docs.colab-finetuning-guide.md)

5. 12개 보호 툴 (function calling)

5.1 핵심 6개 (검증 & 대응)

1. `notify_trusted_contact` — 등록된 가족에게 푸시 알림
2. `suggest_callback` — 저장된 진짜 번호로 다시 걸기
3. `generate_secret_question` — 진짜 가족만 아는 확인 질문
4. `start_wait_timer` — 송금 전 2분 대기 타이머
5. `create_incident_report` — 사건 기록 저장
6. `block_payment_intent` — 송금 차단 하드 게이트

5.2 제2단계 (평가 단계)

1. `block_phone_number` — 전화번호 차단 + 신고 ID
2. `block_email_sender` — 이메일 발신자 스팸 필터
3. `check_url_safety` — 유사 도메인 탐지 (`paypal.com`, `chase-secure.xyz`)
4. `verify_image_message` — 이미지 OCR 텍스트 재분석
5. `show_official_contact` — 사칭당한 브랜드의 진짜 연락처
6. `flag_red_phrases` — 위험 문구를 원문에 하이라이트

브랜드별 공식 연락처: Chase, USPS, IRS, Amazon, FedEx, UPS, PayPal, Wells Fargo, Social Security, Bank of America.

6. RAG 상세 사양

6.1 벡터 DB: ChromaDB 1.5.8

- 로컬 파일 기반 (`data/vector_store/`)
- 별도 서버 인프라 불필요
- [backend/rag.py:23](#)

6.2 임베딩 모델: ChromaDB 기본 임베더

- 내부적으로 `all-MiniLM-L6-v2` (384차원, ONNX 변환본) 사용
- Hugging Face의 `sentence-transformers` 표준 모델
- Windows에서 GPU 충돌 회피 목적

6.3 유사도 함수

- 명시적으로는 L2 거리 (ChromaDB 기본값)
- 하지만 `all-MiniLM-L6-v2` 임베딩은 unit-normalized (벡터 크기 = 1)

단위 벡터에 식는 $\|a-b\|^2 = 2 - 2 \cdot \cos(a,b)$ → L2와 cosine은 단위 상

- 즉 실질적으로 cosine similarity와 같은 결과 (점수 값만 다름)

6.4 현재 비활성화 이유

300개 평가에서 RAG 켜면 FPR 33.1% → 58.9%로 폭증 → 프로덕션 OFF. 이유: 인덱스가 FTC 사기 사례 위주라 평범한 ham 메시지에 사기 사례가 잘못 매칭됨. 재활성화 조건: 인덱스를 카테고리 균형 잡힌 상태로 재구축.

7. 프론트엔드 UI — 폴스크린 차단 오버레이

7.1 변경 전 vs 후

Before: 분석 결과가 SMS 채팅 내 답글로 표시 → URL이나 위험 행동이 여전히 클릭 가능 **After:** 분석 결과가 폴스크린 차단 오버레이로 등장 → 뒤 화면 흐릿, 클릭 불가

7.2 구조



7.3 위험도별 테마 차등화

위험도	헤더 색상	헤더 문구	버튼
safe	초록 (emerald-900)	"Scam Sentinel — scanned, looks normal"	"Continue"
low/medium/ high/critical	빨강 (red-950)	"Scam Sentinel is blocking this content"	"I understand the risk — dismiss" + "We recommend deleting the message instead."

7.4 기술 디테일

- `absolute inset-0 z-40 bg-black/80 backdrop-blur-md` — 시각 차단 + 클릭 차단
- `useEffect(() => setWarningDismissed(false), [result])` — 새 분석마다 자동 리셋
- `lucide-react`의 `<ThumbsUp />` / `<ThumbsDown />` `strokeWidth 1.75`
- 피드백 클릭 → `POST /feedback` → 백엔드 `data/user_feedback.jsonl` append
- `Status Bar / Dynamic Island / Home Indicator`는 변경 없음 (실제 iPhone 느낌 유지)

8. Self-Improving Cascade

해커톤의 핵심 차별화 포인트 — 다른 팀은 정적 모델, 우리는 살아있는 시스템.

8.1 Loop A — Constitutional Self-Critique (이미 구현됨)

목표: 사용자 🗣 데이터로 Gemma 4가 스스로 프롬프트를 고침. 가중치 변경 없음. 실행 시점: 매일 밤 또는 수동, GPU 불필요, 5분 스크립트: [scripts/self_critique.py](https://github.com/0x00sec/scripts/blob/main/self_critique.py)

9단계 동작

1. 트리거 — `python scripts/self_critique.py --apply`
2. 피드백 수집 — `data/user_feedback.jsonl`에서 최근 N개 `false_alarm` 중 non-safe 예측만 필터
3. Critique 프롬프트 빌드 — 현재 `system_prompt` 전문 + N개 false alarms + 제약 조건
4. Gemma 4가 후보 프롬프트 작성 — 제약 위반 시 `NO_REVISION` 출력 강제
5. Stratified eval 샘플링 — 300개 중 라벨 비율 유지하며 50개 선택 (기본)
6. A/B 평가 — 현재 vs 후보 같은 샘플로 추론 → F1/FPR 계산
7. 승급 판정 — $F1 \geq \text{현재}$ AND $FPR < \text{현재}$ 둘 다 만족해야 PROMOTE
8. 승급 실행 (--apply 시) — 기존 프롬프트 아카이브 + 후보로 덮어쓰기 + 로그 append

8.1 단계 18 항상 docs/self_critique_runs/run_<timestamp>.json

안전장치 5중

위험	방어
Gemma 4가 프롬프트 망가뜨림	NO_REVISION 출력 강제 + 스카마/구조 변경 금지
후보가 진짜 사기 놓침	F1 동일 이상 조건
후보가 정상 메시지 잘못 차단	FPR 감소 조건
잘못 채택돼도 복구	기존 프롬프트 자동 아카이브 → 한 줄로 롤백
매 실행 흔적 추적	모든 결과 docs/self_critique_runs/ 저장

8.2 Loop B — DPO Preference Pairs (아직 미구현)

목표: 🇺🇸/🇰🇷 데이터로 LoRA 어댑터 학습. 모델 가중치 변경 (단, base는 그대로, 어댑터만). 실행 시점: 주 1회, Colab L4, 3시간 조건: 🇰🇷 ≥ 200 + 🇺🇸 ≥ 200

8단계 동작

1. Preference pair 빌드 (scripts/build_dpo_pairs.py, 작성 예정)

- 🇰🇷: chosen = safe JSON, rejected = 실제 출력
- 🇺🇸: chosen = 실제 출력, rejected = 가상의 safe JSON

2. Colab으로 데이터 이동 (Drive zip 또는 VS Code Colab extension)

3. Colab L4 노트북 실행 — Unsloth + 4-bit QLoRA

4. 어댑터만 저장 — models/gemma3-scsm-dpo-adapter/ (~50MB)

5. Ollama에 어댑터 등록 — Modelfile에 ADAPTER 지시어

6. 300개 풀 평가 — python scripts/evaluate.py --model gemma3-scsm-dpo

7. 채택 판정 — F1↑ AND FPR↓ 둘 다 만족 시 어댑터 swap

8. 롤백 안전장치 — base는 절대 변경 안 함, Modelfile에서 ADAPTER 줄만 지우면 복원

8.3 Loop A vs Loop B 비교

항목	Loop A	Loop B
무엇이 바뀌나	system_prompt.md 텍스트	LoRA 어댑터 가중치
모델 파라미터	안 바뀜	LoRA 부분만 바뀜 (base 그대로)
GPU 필요?	✗	✓ (Colab L4)
소요 시간	5~30분	2~5시간

비용	\$0	~\$2 (L4 티켓)
빈도	매일	주 1회
개선 범위	표면적 패턴	모델 calibration 자체
위험도	매우 낮음	중간 (forgetting 가능)
데이터 요구	5~10 false alarms	200+ pairs

8.4 함께 돌릴 때 시너지

- **Loop A** 단독: 빠르고 안전하지만, 프롬프트만으로 못 고치는 깊은 편향 해결 못 함
- **Loop B** 단독: 강력하지만 작은 조정에도 매주 3시간 학습 필요
- 둘 다: Loop A가 일상 표면 개선 매일, Loop B가 누적된 깊은 패턴 주말 정리

9. Loop A 데모용 시딩 스크립트

[scripts/seed_demo_feedback.py](#) — 작성 완료, 실행 완료.

9.1 동작

1. 7개 borderline-normal 시나리오를 정의 (정상 메시지만 사기 단어 포함)
2. 각각을 라이브 백엔드 `/analyze/text`로 보냄
3. 결과가 non-safe면 자동으로 `POST /feedback (verdict=false_alarm)`
4. (옵션) 실제 사기 2개도 보내고  자동 기록
5. 통계 + Loop A 실행 가이드 출력

9.2 실행 결과 (2026-05-11)

#	시나리오	모델 예측	정답	결과
1	"Dad, \$300 student account by tomorrow"	medium (urgency+new_account)	safe	✗ 
2	"Mom, phone dying, Venmo \$50 for Uber"	high (urgency)	safe	✗ 
3	"Mom, lost wallet at LAX, \$200 for hotel"	critical (urgency+impersonation)	safe	✗ 
4	"Mom, don't tell Dad — surprise party \$150"	medium (urgency+secrecy)	safe	✗ 
5	"Dad, \$40 for groceries, Sunday"	safe ✓	safe	OK
6	"Team, urgent move standup to 8am"	low (urgency)	safe	✗ 

7	USPS legit shipping notification	safe ✓	safe	OK
+1	"Tyler car accident \$1500" (실제 사기)	high ✓	scam	👍
+2	"Chase verify chase-secure-verify.com" (실제 사기)	medium ✓	scam	👍

결과 요약: 7개 중 5개 false alarm 발생 = **71%** 오탐률 (300개 평가셋의 33%보다 더 심함, 일부러 borderline 만 골랐기 때문)

9.3 누적된 피드백 파일 상태

[data/user_feedback.jsonl](#) 총 9행:

- 🔊 false_alarm: 6개 (Loop A 트리거 조건 ≥3개 통과 ✓)
- 👍 correct: 3개
- false_alarm 위험도 분포: high 2 / medium 2 / critical 1 / low 1

10. Loop A 데모 영상 캡처 계획 (3분)

10.1 영상 구성 (3분 제안)

구간	시간	내용
0:00–0:20	20s	문제 hook (피싱 문자/전화/이메일)
0:20–0:35	15s	앱 한 줄 소개 + UI 잠깐 보여주기
0:35–1:00	25s	모델 & 메트릭 (텍스트 카드 + 핵심 숫자)
1:00–1:50	50s	☆ 실시간 데모 — 시나리오 2개
1:50–2:35	45s	Self-Improving Cascade (Loop A 동작 + Loop B 한 줄)
2:35–3:00	25s	임팩트 마무리

10.2 데모 시연 단계

1. 사전 점검: 백엔드 + 프론트엔드 서버 둘 다 실행 중

2. **Before** 캡처 (3분, 영상 안):

- 브라우저 <http://localhost:3000>
- ✓ **Normal family message** 시나리오 → Analyze → 폴스크린 빨간 경고 (false alarm) → 🔊 클릭

3. 데이터 누적 보여주기: [data/user_feedback.jsonl](#) 보여주기

3. **on-device** 데모 — LiteRT 또는 Cactus로 폰에서 직접 실행 (Special Tech Track \$10K)

4. 다국어 지원 — 한국어 데모 (subtitles only, 모델은 English-only 유지)

12. 핵심 파일 구조

```
scam-sentinel/
├─ CLAUDE.md                source of truth for design
├─ README.md               public-facing (with eval results)
├─ docs/
│   └─ architecture-prompt.md    self-contained system brief
│   └─ conversation-summary.md   ← 이 문서
│   └─ eval_results.md           full eval methodology
│   └─ prompt_versions.md        v1 → v2 → v3 history + Loop A logs
│   └─ colab-finetuning-guide.md  Colab Pro L4 step-by-step
├─ backend/
│   └─ main.py               FastAPI: /analyze/*, /feedback
│   └─ reasoning_agent.py    Gemma 3 + Gemma 4 cascade
│   └─ tools.py              12 protective tools
│   └─ rag.py                ChromaDB retriever
│   └─ prompts/
│       └─ fast_classifier.md    Gemma 3 prompt
│       └─ system_prompt.md     Gemma 4 prompt (v3)
│       └─ synthesis.md         synthetic-data prompt
├─ frontend/app/
│   └─ page.tsx
│   └─ components/
│       └─ PhoneEmulator.tsx    iPhone shell + FullScreenWarning
│       └─ AnalysisPanel.tsx    CoT reasoning panel
│       └─ InputForm.tsx
├─ data/
│   └─ seeds.jsonl (80)
│   └─ seeds_real.jsonl (571)
│   └─ synthetic/ (train 3,100 / dev 771 / raw etc.)
│   └─ evaluation/ (eval_set 300 + eval_set_70 70)
│   └─ rag_cases.jsonl (117)
│   └─ vector_store/          ChromaDB persistent index
│   └─ user_feedback.jsonl    Self-Improving Cascade input (9 rows)
├─ scripts/
│   └─ generate_synthetic.py
│   └─ extract_real_seeds.py
│   └─ filter_quality.py
│   └─ expand_eval_set.py
│   └─ evaluate.py
│   └─ self_critique.py       Loop A ✓
│   └─ seed_demo_feedback.py  Loop A 데모 시딩 ✓
│   └─ build_dpo_pairs.py     Loop B (작성 예정)
│   └─ prepare_finetune_data.py
│   └─ train_lora.py
```

13. 한 줄 요약

메시지 입력 → Gemma 3로 빠른 분류(safe면 끝) → 위험하면 Gemma 4가 5단계 추론으로 사기 패턴 분석 → 12개 보호 톨 자동 실행 → 폴스크린 차단 오버레이로 URL앱 클릭 방지 → 사용자 피드백 → Loop A(매일, 프롬프트 자기개선) + Loop B(주 1회, LoRA 학습)로 시스템 자동 진화. 다른 팀이 정적 모델을 제출할 때, 우리는 살아있는 시스템을 제출한다.

문서 끝. 이 문서는 처음부터 끝까지 한 번에 읽을 수 있도록 구성되었으며, docs/architecture-prompt.md와 함께 보면 모든 설계 결정의 맥락을 파악할 수 있습니다.